

Artificial Stupidity Human Companion

VERSION 0.9.4 • CLAIM PRECISION EDITION

Raymond Anthony Gomez | September 15, 2026



Intelligence proposes. A separate gate challenges. Evidence informs the decision. A human retains final authority.

CURRENT STATUS Working research prototype. Human external review submitted; findings pending. Independent reproduction has not been completed.

Start here

Artificial Stupidity is a working research prototype that places a separate acceptance boundary around an AI research loop. Its purpose is simple: an AI may propose an improvement, but a promising score does not automatically earn approval.

The prototype can return **KEEP**, **REJECT**, or **ESCALATE** under declared rules. It has 74 automated CPU tests, a public demonstration, preserved source and evidence, a recorded six-run H100 experiment, disclosed failures and repairs, and a reviewer packet designed for an outsider to inspect. It has not completed independent reproduction, peer review, safety validation, or production validation. A human external review has been requested and remains pending.

The most important lesson is not that one model setting was proven better. It was not. The important result is that the system can separate a machine's proposal from the decision to accept it—and can stop when its evidence or authority runs out.

The idea in plain English

Imagine an AI system changes a piece of code, runs an experiment, and produces a better number. The number may be real. It may also be fragile, incomplete, confounded, or optimized in a way that misses the actual purpose.

Artificial Stupidity adds a separate checkpoint. The checkpoint asks whether the evidence meets rules established outside the proposing loop. It can approve a bounded machine-level decision, reject a known failure, or stop and ask a person to decide.

The name began as a joke, but the distinction became serious:

The project does not make AI less intelligent. It makes acceptance harder to earn.

What happened

A bruised ego and a joke

The project began after an AI comparison pointed Ray toward Andrej Karpathy's [autoresearch](#) repository as an example of stronger autonomous AI experimentation. Karpathy coined the phrase “vibe coding,” and his public project offered a compact research loop: modify a training file, run a time-bounded experiment, measure the result, and keep or discard the change.

The comparison stung. The productive response was not to argue with it. Ray downloaded the model, studied what it did, and asked a different question:

Should the system proposing an improvement also be allowed to decide that the improvement is acceptable?

A Harry Potter joke about “Artificial Stupidity” supplied a memorable name. A historical search then showed that the phrase and its underlying concerns were not original. Earlier uses often involved intentional machine mistakes, humanlike imperfection, limits on capability, or the difference between calculation and judgment. This project took a narrower path: preserve the intelligent engine and add a separate, inspectable acceptance boundary around its output.

A working gate

The prototype added decision logic outside the optimizer's editable training file. The gate evaluates declared evidence and returns one of three actions:

Action	Plain meaning
KEEP	The candidate met the rules the machine was authorized to evaluate.
REJECT	The candidate hit a declared disqualifying condition.
ESCALATE	The result may be promising, but evidence or machine authority is insufficient.

Machine decisions and human authorization are stored separately. A human can review an escalated result, but the human decision does not rewrite the machine's earlier record.

The first experiment stopped

An early H100 experiment tested a one-line configuration change: total batch size was reduced from 2**19 to 2**18. The ordinary validation measure improved, but the evidence package did not yet include a separately stored holdout evaluation. The gate returned **ESCALATE**.

That was a useful result. The system did not treat “the score improved” as enough evidence for automatic acceptance.

The six-run H100 record

A later run used three baseline and three candidate experiments, interleaved in this order:

B1 / C1 / B2 / C2 / B3 / C3

Under that fixed-wall-clock setup, all three candidate runs recorded lower validation bits per byte than all three baseline runs. The same ordering appeared on the separately stored holdout. Comparing every candidate with every baseline produces nine cross-arm comparisons on each measure.

Metric	Baseline median	Candidate median	Recorded difference
Validation BPB	1.000213	0.990934	0.928% lower
Sequestered-holdout BPB	0.999357	0.990126	0.924% lower
Peak VRAM	45,060.2 MB	44,908.2 MB	0.337% lower

These are **six runs**, not nine independent trials. The nine figures are all-by-all comparisons formed from the three runs in each arm.

The experiment also leaves an important causal question unresolved. Each run received the same wall-clock budget, but optimizer steps and tokens processed were not held constant. Halving batch size may have changed how many updates or tokens each arm completed. Therefore, this record does not establish that the smaller batch size caused the measured difference.

The safe conclusion is narrower:

Under this one disclosed fixed-time setup, this candidate recorded lower validation and holdout BPB than the baseline in the six preserved runs.

The machine still escalated

The gate was not allowed to certify that the change was understandable to a responsible human. Human comprehensibility was intentionally left unknown, so the machine returned **ESCALATE** even after the recorded metric improvements.

Ray then reviewed the one-line change and the bounded evidence and separately authorized **KEEP**. This was author authorization, not independent reproduction or outside validation. Its value lies in the visible separation of roles: the machine reached its authority boundary, stopped, and preserved the decision for a named person.

A late failure became evidence

After the paid training runs and evaluations completed, the automation encountered an import-path error while reaching its final decision step. The raw logs, source snapshot, hashes, environment record, and partial outputs had already been preserved.

The decision was reconstructed from the captured evidence without additional model training. The import path was repaired, a regression test was added, and the original failure remained public.

This did not prove that the overall system was production-ready. It demonstrated something more limited and useful: evidence preservation made a late automation failure inspectable and repairable without pretending it never happened.

Adversarial testing changed the gate

The first frozen adversarial campaign exposed a serious trust-boundary weakness. Eight of nine attack probes produced unjustified **KEEP** outcomes in version 0.9.1. Version 0.9.2 repaired those paths, added clearer handling for unknown evidence, and passed the frozen follow-up attack and control matrix.

Both the failure and the repair remain public. The correct lesson is not “the gate is secure.” It is that an explicit attack plan found weaknesses that ordinary happy-path testing had missed, and the project preserved that learning.

A second domain tested the shape of the method

The gate's decision pattern was also exercised on three Bill X-Ray legislative claims. The cases were selected, labeled, and frozen by the same project team before a separate adapter was implemented. The system kept a supported statutory statement, rejected a contradicted mandate-and-funding claim, and escalated an unsupported forecast.

This is a same-team, deterministic, pre-labeled portability exercise. It shows that the declared decision pattern can be mapped to another kind of evidence. It is not independent domain validation, legal advice, an error-rate estimate, or proof that the method generalizes.

Claude challenged the interpretation

After the public launch, Claude was asked to inspect the repository, packet, live demo, portfolio, and claims rather than accept the project's framing. Its critique identified several places where technically true facts could be read too broadly.

The most important challenge was the fixed-wall-clock confound. The experiment had not controlled for total optimizer steps or tokens processed, so the measured difference could not be attributed to batch size alone. Claude also challenged the use of “independent,” the prominence of “9/9,” the strength of the second-domain language, and the difference between a submitted review and a completed one.

Claude served as an **AI adversarial pre-reviewer**. It did not reproduce the experiment, conduct peer review, or provide independent human validation. Its value was practical: it found interpretive weaknesses, and the accepted corrections were checked against the public record.

Version 0.9.4 narrowed the claims without changing the underlying H100 measurements. The prior release remains public as the before-state.

Human review remains pending

The corrected v0.9.4 packet was sent for external human review. That reviewer is outside the project's execution and authorship, but no findings have been received yet.

The current status is therefore exact:

Submitted for independent human review. Review pending.

Submission is not reproduction, endorsement, certification, or validation.

What the project establishes

The current public record supports these bounded statements:

- The gate executes and returns **KEEP**, **REJECT**, or **ESCALATE** under declared rules.
- The gate can withhold automatic acceptance after a metric improves.
- The implementation has 74 automated CPU tests. A passing deterministic suite demonstrates conformance to declared cases, not a real-world safety rate.
- Machine judgment and named human authorization are stored separately.
- The preserved H100 record contains three baseline and three candidate runs under one fixed-time setup.
- Each preserved candidate run recorded lower validation and sequestered-holdout BPB than each preserved baseline run.
- A late import failure, the reconstruction, and the repair remain visible.
- A frozen adversarial campaign exposed serious weaknesses, and the follow-up version repaired the tested paths.
- The same decision pattern produced the three predeclared outcomes in a same-team Bill X-Ray exercise.

What the project does not establish

The record does not support these broader statements:

- That smaller batch sizes generally or causally improve model performance.
- That the measured difference generalizes across models, datasets, hardware, or domains.
- That the holdout was cryptographically secret or protected from every form of leakage or gaming.
- That a passing deterministic test suite predicts a real-world error or safety rate.
- That the Bill X-Ray exercise is independent validation or legal evaluation.
- That producer labels or allowlists provide cryptographic identity.
- That the project has completed independent reproduction, peer review, safety validation, or production validation.
- That human authorization guarantees a correct decision.

How to experience the project

One minute

Open the [live demonstration](#). Choose a supported gain, weak gain, or incomplete-evidence scenario and run the gate. The page calls the same public Python decision function used in the repository; the outcomes are not hard-coded into the page.

Five minutes

Read the current [GitHub README](#), inspect the claim boundary, and review the preserved failure-and-repair chronology.

Serious technical review

Use the frozen [v0.9.4 external-reviewer packet](#). It contains the start guide, reproduction protocol, qualification guide, review checklist, findings template, claim-precision record, hashes, and release references.

Roles and authority

Role	What it may do	What it may not claim
Proposer	Suggest a candidate change and state the intended result.	Approve its own work merely because a score improved.
Evidence harness	Run declared comparisons and preserve measurements, logs, source, and hashes.	Convert measurements into a causal explanation by itself.
Gate	Apply declared rules and return KEEP, REJECT, or ESCALATE.	Exceed its delegated authority or certify human understanding.
AI challenger	Attack logic, wording, assumptions, and missing controls.	Count as independent reproduction or human review.
Human authorizer	Accept responsibility for a bounded decision after reviewing the record.	Turn limited evidence into universal truth.

Role	What it may do	What it may not claim
External reviewer	Reproduce declared procedures and publish findings under a disclosed relationship.	Be described as having validated the project before completing the work.

The next experiment

The highest-value next test is designed to separate a useful engineering result from a causal batch-size claim.

Before any new output is inspected, the protocol should freeze:

- six matched baseline and candidate pairs;
- interleaved execution;
- matched seeds within each pair where the implementation allows;
- different seeds across pairs;
- logged wall time, optimizer steps, and tokens processed for every run;
- an added token-matched control that separates batch size from training exposure;
- a fixed stopping rule;
- raw reporting of wins and losses without turning a small sample into a significance claim;
- independent recomputation of BPB and the gate decision from raw evidence.

The justified result afterward would still be narrow: under a named architecture, dataset, environment, and protocol, the candidate did or did not retain its measured advantage when training exposure was controlled.

This experiment has been proposed. It has not yet been run.

The broader design pattern

The reusable contribution is an **evidence-gated remix**:

1. Begin with a credible, properly licensed engine.
2. Preserve attribution and the upstream boundary.
3. Identify a consequential decision the engine should not make automatically.
4. Put acceptance rules outside the proposing system's authority.
5. Declare the evidence and stopping rules before evaluating outputs.
6. Preserve failures, unknowns, repairs, and human decisions.
7. Invite a challenger to attack the claim as well as the code.
8. Publish only what the evidence earns.

This pattern may be useful in research, coding agents, analytics, compliance workflows, procurement, and other settings where systems can generate persuasive outputs faster than people can responsibly inspect them. Use in any real organization would require domain-specific rules, threat modeling, qualified review, privacy controls, and operational testing.

Why the story matters

Artificial Stupidity began with embarrassment and humor, became a working prototype, suffered failures, gained evidence, overreached in places, and then became more credible by accepting criticism.

That sequence is not clutter around the project. It is the project.

The enduring idea is not that AI should be made foolish. It is that rapid machine intelligence needs a visible place for organized doubt—and that neither a better metric, a polished story, nor an AI's confidence should be mistaken for earned authority.

Current links and status

- **Live demonstration:** <https://artificial-stupidity.onrender.com/>
- **Repository:** <https://github.com/rayoonbaco/artificial-stupidity>
- **Claim-precision release:** <https://github.com/rayoonbaco/artificial-stupidity/releases/tag/v0.9.4-claim-precision>
- **Portfolio:** <https://the-receipts.onrender.com/>
- **Automated CPU tests:** 74
- **Human external review:** Submitted; findings pending
- **Independent reproduction:** Not completed
- **Next controlled experiment:** Proposed; not run

Claim boundary

This companion explains the current public record in plain English. It does not expand the scientific claims of the repository or replace the reviewer packet, raw evidence, source history, or qualified technical review. Historical editions should remain available as dated before-states, but this v0.9.4 edition should be used for current communication.

The claim became smaller. The project became stronger.